

Formalization Seminar: Kickoff

George McNinch

Tufts University

2026-09-15

Welcome	1
Plan for today	2
Proof assistants and Lean	3
Live demo	7
Where this is headed	9

- proof assistant and *Lean*?
- a short live demo – watch Lean check a proof
- where I'd like to head: formalizing some undergraduate analysis
- logistics

Welcome	1
Proof assistants and Lean	3
What is a proof assistant?	4
Why Lean, specifically?	5
A taste of what Lean looks like	6
Live demo	7
Where this is headed	9

What is a proof assistant?

- It is a piece of software for:
 - **defining** mathematical objects,
 - **specifying** properties of those objects,
 - **proving** that the specifications hold
- the system (“proof assistant”) checks that the proof is correct. This check feels a bit like a *compilation step*.
- contrast with automated theorem proving: there, the **computer** searches for the argument. Here, **you** construct it and the proof assistant checks it.
- there are many different examples of proof assistants: Lean, Coq, Isabelle/HOL, Agda, Mizar, ...

The differences mostly come down to the choice of underlying logic.

Why Lean, specifically?

- Lean's foundations – **dependent type theory** (Calculus of Constructions with inductive types) – is expressive enough to state and prove essentially any theorem you'd meet in ordinary mathematics
- There is a large, actively maintained, community library of formalized mathematics known as **mathlib** – see <https://github.com/leanprover-community/mathlib4>
- Mainly, this seminar will take a pragmatic view on this subject. Rather than describing type-theoretic foundations for their own sake – we'll just focus on how to use Lean to formalize mathematics.

```
-- a definition
def append {α : Type} (xs ys : List α) : List α :=
  match xs with
  | []      => ys
  | z :: zs => z :: append zs ys

-- a theorem, and its proof
theorem append_length {α : Type} (xs ys : List α) :
  (append xs ys).length = xs.length + ys.length := by
  induction xs with
  | nil => simp [append]
  | cons z zs ih =>
    simp only [append, List.length_cons]
    rw [ih]
    ring
```

Welcome	1
Proof assistants and Lean	3
Live demo	7
Let's watch Lean check a proof	8
Where this is headed	9

Let's watch Lean check a proof

Live demo

- let's switch to the editor now
- we'll prove `append_length` from scratch, live
- watch for what happens if a step is **wrong**?

Welcome	1
Proof assistants and Lean	3
Live demo	7
Where this is headed	9
The target: formalizing undergraduate analysis	10
Suggested reading	11

The target: formalizing undergraduate analysis

Where this is headed

- the plan for this seminar: work, together, toward formalizing pieces of the undergraduate analysis course
- here's the kind of statement we mean – convergence of a sequence:

-- "u tends to ℓ "

```
def seq_limit (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) ( $\ell$  :  $\mathbb{R}$ ) :=  
   $\forall \varepsilon > 0, \exists N, \forall n \geq N, |u\ n - \ell| \leq \varepsilon$ 
```

- and a real theorem about it – the squeeze theorem:

```
theorem squeeze (hu : seq_limit u  $\ell$ ) (hw : seq_limit w  $\ell$ )  
  (h :  $\forall n, u\ n \leq v\ n$ ) (h' :  $\forall n, v\ n \leq w\ n$ ) :  
  seq_limit v  $\ell$  := by  
  sorry
```

- from *Mathematics in Lean*:
 - §2.3 (“Using Theorems and Lemmas”) is the most useful part of Ch. 2 for where we’re headed – the rest of Ch. 2 you can skim or skip.
 - Ch. 3 (Logic), through §3.6 (“Sequences and Convergence”)
 - Ch. 4 (Sets and Functions)
- try some of the exercises as you go